



TECNOLÓGICO
NACIONAL DE MÉXICO®



Tecnológico Nacional de México

Instituto Tecnológico de Culiacán

Ingeniería en sistemas computacionales

Tópicos de Inteligencia Artificial

10:00 – 11:00

Proyecto Modulo 4

Visión Artificial – Detección de Rostros con Asociación de Datos

Equipo:

Zavala Carmona Miguel Angel

Zazueta Torres Yahir Alexander

30/05/2026

Link al repositorio de GitHub: <https://github.com/MiguelZC20/Topicos-de-Inteligencia-Artificial.git>

Contenido

Introducción	3
1. Descripción del proyecto	3
1.2 Objetivo general	3
1.3 Objetivos específicos	3
Desarrollo.....	3
2. Recolección de Datos.....	3
2.1. Etiquetado	4
2.2. División del Conjunto de Datos.....	5
2.3. Preprocesamiento de los Datos	5
3. Diseño e Implementación del Modelo.....	6
3.1. Transfer Learning.....	6
3.2. Selección de MobileNetV2	7
3.3. Arquitectura del modelo	7
4. Entrenamiento y Validación del Modelo	8
5. Conversión del Modelo para Dispositivos Móviles.....	10
6. Desarrollo e Integración de la Aplicación Móvil.....	11
6.1 Selección de la Plataforma de Desarrollo	11
6.2 Configuración del Entorno de Desarrollo	11
7. Implementación del Sistema de Detección y Reconocimiento de Rostros	13
7.1. Flujo general del reconocimiento.....	13
8. Resultados Experimentales y Evaluación del Sistema	14
9. Conclusiones.....	18

Introducción

1. Descripción del proyecto

Desarrollar una aplicación móvil capaz de identificar rostros de alumnos utilizando modelos de entrenamiento y técnicas de visión artificial, y mostrar posteriormente información asociada a cada alumno.

1.2 Objetivo general

Diseñar e implementar un sistema de reconocimiento de rostros basado en aprendizaje profundo capaz de identificar alumnos mediante una aplicación móvil Android.

1.3 Objetivos específicos

- Construir un conjunto de datos de rostros de alumnos.
 - Entrenar un modelo de reconocimiento facial.
 - Integrar el modelo entrenado en una aplicación para dispositivos móviles.
 - Mostrar información del alumno reconocido.
-

Desarrollo

2. Recolección de Datos

La primera etapa del proyecto consistió en la obtención de un conjunto de datos de los rostros de alumnos para el sistema de reconocimiento. Para ello, se solicitó la colaboración de los alumnos para que proporcionaran fotografías de sus rostros tomadas desde diferentes ángulos. De igual manera se recabo información escolar de cada alumno para asociarla con la detección de su rostro.

El conjunto de datos inicial estuvo conformado por 16 alumnos, recopilando entre 8 y 10 imágenes por persona, obteniendo aproximadamente 136 imágenes.

Una vez obtenidas las imágenes, estas fueron organizadas en una estructura jerárquica de carpetas, donde cada carpeta represento a una clase específica. En otras palabras, cada alumno tiene su carpeta con sus respectivas imágenes.

La organización inicial quedo entonces de esta forma:

data -> raw ->

- alumno_01
- alumno_02
- alumno_03
-

Características del Dataset original:

Concepto	Descripción
Alumnos	16 (Reales)
Imágenes por alumno	8-10
Total	136 imágenes
Formato	JPG/JPEG
Etiquetado	Por carpeta

2.1. Etiquetado

En problemas de clasificación supervisada es importante que los datos de entrenamiento estén asociados a una etiqueta que identifique a la clase a la que pertenece. En este caso, era importante para saber a qué alumno pertenecen las imágenes.

En este proyecto se utilizo entonces una estrategia de **etiquetado implícito** basada en directorios. Es decir, el nombre de la carpeta que contiene las imágenes de cada alumno se considera la etiqueta correspondiente a dicha muestra. Por ejemplo, todas las imágenes dentro de la carpeta “alumno_01” fueron asociadas a esa clase.

Posterior a esto, se generó una asociación numérica entre cada alumno y un identificador, necesaria para el entrenamiento del modelo de aprendizaje profundo.

La codificación fue entonces de la siguiente forma:

Alumno	Identificador
alumno_01	0
alumno_02	1
alumno_03	2
.....	

2.2. División del Conjunto de Datos

Una vez etiquetadas las imágenes, se procedió a dividir el conjunto de datos en subconjuntos para entrenamiento, validación y pruebas.

Teniendo en cuenta la cantidad de imágenes por alumno en el Dataset original se decidió dividir los datos con las siguientes proporciones:

- 70 % para entrenamiento.
- 15 % para validación.
- 15 % para pruebas.

La distribución se realizó manteniendo una proporción de imágenes similar para cada alumno, evitando así un desbalance en las clases. Al final, la estructura obtenida fueron directorios (entrenamiento, validación, pruebas) con subcarpetas correspondientes a cada alumno.

2.3. Preprocesamiento de los Datos

Detección y Recorte de Rostros

Las fotografías obtenidas originalmente contenían información adicional irrelevante, como fondos, objetos o partes del cuerpo que no fuese el rostro.

Para minimizar este problema, se implementó un proceso automático de detección y recorte facial utilizando técnicas de visión artificial basadas en **OpenCV**.

Para la detección de los rostros se utilizó el clasificador **Haar Cascade** denominado **“haarcascade_frontalface_default.xml”**.

El clasificador permitió localizar automáticamente la región facial dentro de cada imagen y generar una nueva imagen únicamente con el rostro detectado.

Como resultado, se obtuvo un conjunto de imágenes centradas únicamente en el rostro de los alumnos. Se realizó de igual forma una inspección manual de las imágenes generadas con el objetivo de encontrar recortes incorrectos y eliminarlos para evitar que afectaran después en el entrenamiento del modelo.

Redimensionamiento

Las redes neuronales convolucionales (CNN) requieren que todas las imágenes de entradas tengan dimensiones uniformes. Sin embargo, las fotografías capturadas presentaban diferentes resoluciones debido al dispositivo con el que fueron tomadas. Para solucionar este problema, todas las imágenes fueron redimensionadas a un tamaño de 160 x 160 píxeles.

Esta resolución fue seleccionada teniendo en cuenta los requisitos del ya contemplado modelo **MobileNetV2** para usarse posteriormente en el entrenamiento.

Data Augmentation

Uno de los desafíos del proyecto fue el tamaño relativamente pequeño del conjunto de datos. A pesar de disponer de múltiples fotografías de cada alumno, la cantidad total seguía siendo pequeña.

Para reducir este problema se implementó el “Data Augmentation”, una técnica que consiste en generar variaciones artificiales de las imágenes originales mediante transformaciones.

Durante el entrenamiento se aplicaron las siguientes transformaciones:

- **Rotación aleatoria:** para simular pequeñas inclinaciones con la cabeza.
- **Zoom aleatorio:** simular variaciones de distancia entre la persona y la cámara.
- **Contraste aleatorio:** simular variaciones de luz.
- **Inversión horizontal:** genera una versión reflejada, como si fuera un espejo.

Es importante mencionar que en este proyecto el Data Augmentation no incremento la cantidad de imágenes almacenadas en el Dataset. Lo que ocurrió es que con ayuda de **TensorFlow** se generaron las versiones modificadas durante el entrenamiento.

Esto quiere decir que una misma imagen podía presentarse durante una época con una ligera rotación, mientras que en una época posterior aparecer con una inversión horizontal o diferente contraste.

3. Diseño e Implementación del Modelo

Una vez completadas las etapas anteriores, se procedió al diseño e implementación del modelo encargado de realizar la identificación de los alumnos a partir de sus rostros.

3.1. Transfer Learning

Debido al tipo de problema y la cantidad limitada de imágenes disponibles para entrenamiento, se optó por utilizar el transfer learning y aprovechar modelos ya entrenados con grandes conjuntos de datos.

La idea es que las primeras capas de una red convolucional aprendan características visuales generales (bordes, líneas, curvas, formas) y adaptar únicamente las capas finales para resolver una tarea específica.

3.2. Selección de MobileNetV2

Se seleccionó la CNN MobileNetV2 desarrollada por Google debido a su equilibrio en precisión, velocidad, consumo y compatibilidad con dispositivos móviles.

Era la mejor opción para una aplicación Android basada en TensorFlow.

3.3. Arquitectura del modelo

MobileNetV2

Actúa como el extractor automático de características faciales. Su función consiste en transformar la imagen en una representación matemática que capture información relevante del rostro.

Global Average Pooling

Reduce la dimensionalidad de la representación generada por MobileNetV2. Esto hace que disminuya la cantidad de parámetros necesarios en las capas posteriores.

Capa densa de 180 neuronas

Aprende combinaciones específicas de características faciales relevantes para diferenciar entre los alumnos.

Dropout

Se incorporó una capa Dropout como técnica que desactiva aleatoriamente neuronas de la red durante el entrenamiento. Esto evita que el modelo dependa excesivamente de características específicas y desarrolle representaciones más robustas.

Capa Softmax

La última capa que contiene 16 neuronas, una por cada alumno. Softmax transforma la salida de la red en probabilidades asociadas a cada clase.

Por ejemplo:

alumno_01 -> 0.02

alumno_02 -> 0.91

alumno_03 -> 0.00

....

En este caso, el sistema estaría seleccionando al alumno_02 como resultado final.

4. Entrenamiento y Validación del Modelo

Para entrenar la red neuronal se utilizaron los siguientes parámetros

Parámetro	Valor
Tamaño de entrada (imagen)	160 x 160
Batch Size	8
Optimizador	Adam
Función de pérdida	Sparse Categorical Crossentropy
Épocas	20

Optimizador Adam

Fue seleccionado debido a su capacidad para ajustar automáticamente la tasa de aprendizaje durante el entrenamiento. Este optimizador ofrece una convergencia rápida y estable.

Función de pérdida

Se utilizó Sparse Categorical Crossentropy debido a que las clases fueron representadas mediante identificadores numéricos enteros.

Monitoreo del entrenamiento

Durante el entrenamiento se registraron dos métricas principales:

- **Accuracy:** permitió evaluar el porcentaje de clasificaciones correctas realizadas por el modelo.
- **Loss:** permitió medir el error cometido durante el proceso de aprendizaje.

Resultados del entrenamiento

Una vez concluido el proceso de aprendizaje, el modelo fue evaluado utilizando el conjunto de prueba reservado anteriormente.

Obteniendo como resultado un Accuracy de prueba de 93.75 %.

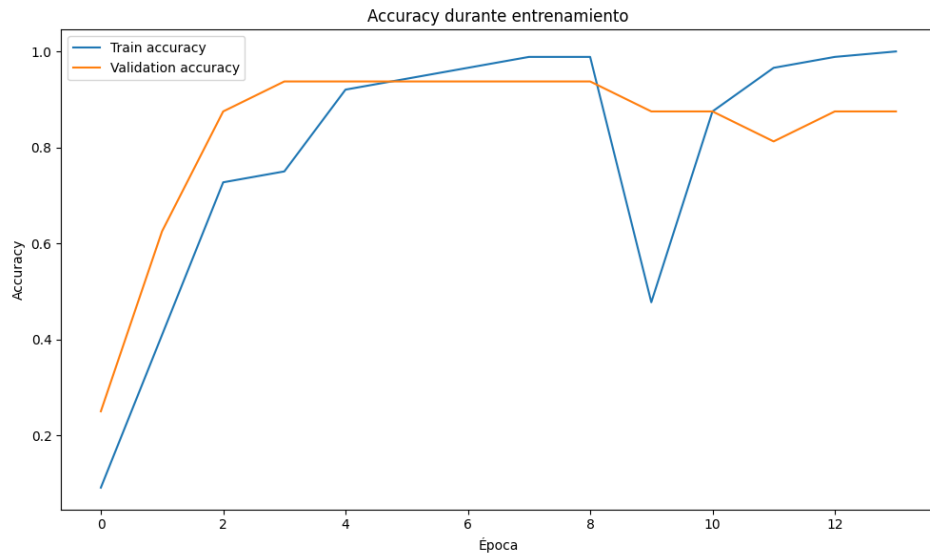


Ilustración 1 Curva de "Accuracy" durante entrenamiento

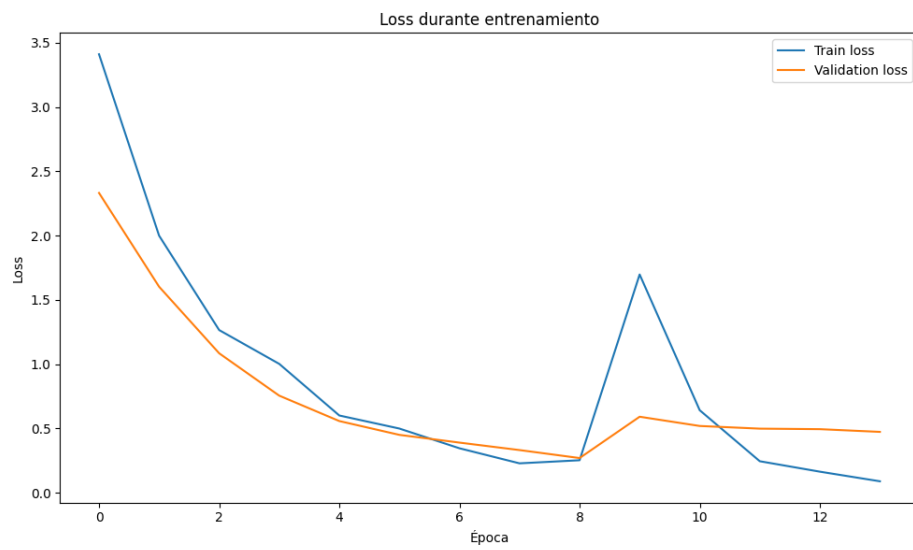


Ilustración 2 Curva de "Loss" durante entrenamiento

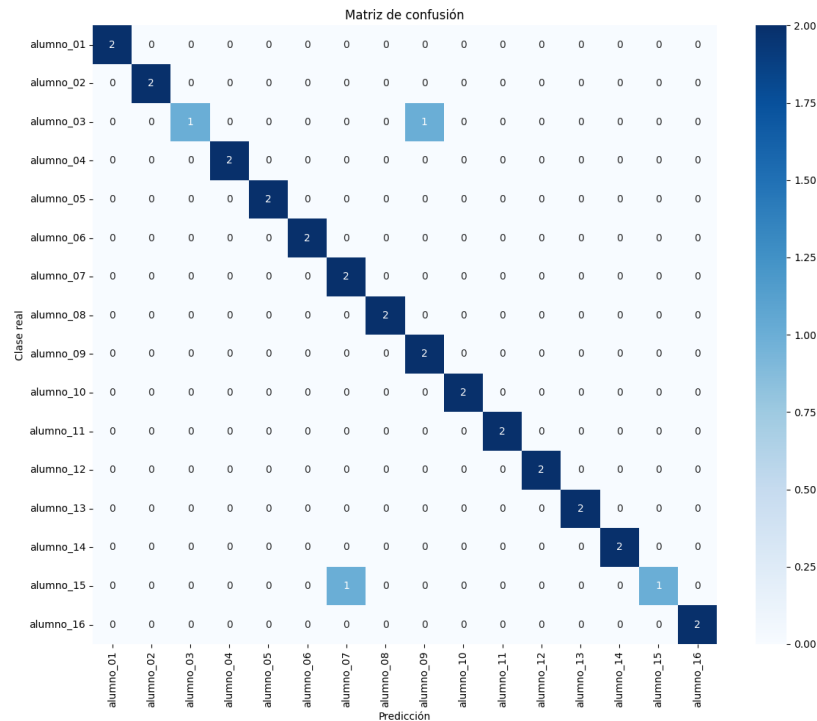


Ilustración 3 Matriz de confusión

Estos resultados indican que el modelo logró aprender correctamente patrones faciales distintos para la mayoría de los alumnos incluidos en el Dataset.

La matriz de confusión generada permitió identificar algunas clases con mayor dificultad de reconocimiento, proporcionando información útil para futuras mejoras.

Estos resultados dejan en claro la viabilidad de utilizar técnicas de transfer learning para desarrollar sistemas de reconocimientos de rostros incluso cuando se tienen conjuntos de datos pequeños.

5. Conversión del Modelo para Dispositivos Móviles

Los modelos de aprendizaje profundo son entrenados en computadoras con mayores capacidades de procesamiento y memoria que las de los dispositivos móviles.

Aunque el modelo entrenado logro resultados satisfactorios durante las pruebas en el entorno de desarrollo, su despliegue en una aplicación Android requería de una representación adecuada para que pudiera ser ejecutada.

TensorFlow Lite

Es una plataforma desarrollada por TensorFlow específicamente para la ejecución de modelos de aprendizaje en dispositivos móviles. Por esto mismo, fue seleccionado como mecanismo de despliegue del modelo.

Conversión del modelo

Una vez finalizado el entrenamiento, el modelo fue almacenado en formato keras. Posteriormente se utilizó el convertidor oficial de TensorFlow Lite para generar una versión optimizada del modelo.

Como resultado, se obtuvo un archivo con un tamaño aproximado de 2.43 MB, bastante adecuado para aplicaciones móviles sin afectar al dispositivo.

6. Desarrollo e Integración de la Aplicación Móvil

6.1 Selección de la Plataforma de Desarrollo

Para la implementación de la aplicación móvil se seleccionó Flutter debido a las ventajas que ofrece para aplicaciones que integran componentes de IA y visión artificial.

6.2 Configuración del Entorno de Desarrollo

Para el desarrollo de la aplicación se configuró un entorno con diversas herramientas específicas.

Flutter SDK

Se instaló como el framework principal para el desarrollo de la aplicación, permitiendo:

- Herramientas de compilación.
- Emulación.
- Gestión de dependencias.

Android Studio

Fue utilizado para:

- Gestión del SDK de Android.
- Creación de emuladores.
- Herramientas de compilación.

Creación de la aplicación Flutter

La aplicación fue creada utilizando la estructura estándar proporcionada por Flutter.

Mediante el comando “flutter create mobile” se generó una estructura base, donde posteriormente se incorporaron los recursos necesarios para el sistema de reconocimiento.

Organización del proyecto

Se adoptó una estructura modular para separar por funciones específicas. Dentro de la estructura principal del proyecto se organizó de la siguiente manera:

- **database:** contiene la lógica relacionada con el almacenamiento local de información de alumnos usando SQLite.
- **vision:** se agrupan los componentes relacionados con la detección de rostro y reconocimiento mediante TensorFlow Lite.
- **main:** implementa la interfaz principal de la aplicación y coordina la comunicación entre los módulos.

Dependencias

Para implementar diferentes funcionalidades fue necesario incorporar bibliotecas externas:

Dependencia	Función
camera	Acceso a la cámara del dispositivo
google_mlkit_face_detection	Detección facial
tflite_flutter	Ejecución del modelo TensorFlow Lite
image	Procesamiento de imágenes
sqlite	Base de datos SQLite
path_provider	Gestión de directorios locales
path	Manipulación de rutas

7. Implementación del Sistema de Detección y Reconocimiento de Rostros

7.1. Flujo general del reconocimiento

Inicialización de la cámara

La primera tarea que realiza el sistema es obtener acceso a la cámara frontal del dispositivo, permitiendo observar la imagen a capturar antes de iniciar el reconocimiento.

Detección y Extracción Facial mediante Google ML Kit

Una vez capturada la imagen, es necesario localizar el rostro. Para ello se utilizó el ML Kit de Google, el cual recibe una imagen capturada por la cámara y ejecuta un detector facial que localiza la ubicación del rostro.

Una vez localizado el rostro, el sistema extrae únicamente la “región facial” que será enviada al modelo entrenado.

Reconocimiento facial

Es uno de los componentes del módulo “vision” responsable de:

- Cargar el modelo TensorFlow Lite.
- Preparar la imagen de entrada.
- Ejecutar inferencia.
- Interpretar resultados.

Preprocesamiento

Antes de enviar la imagen al modelo, es necesario aplicar el mismo preprocesamiento utilizado durante el entrenamiento. Las principales transformaciones realizadas son:

- Redimensionamiento (imagen 160 x 160 píxeles).
- Conversión de píxel a valores numéricos compatibles con TensorFlow Lite.
- Normalización de valores esperados por MobileNetV2.

Ejecución

Una vez preparada la imagen, esta es enviada al modelo TensorFlow Lite y como resultados obtenemos un vector de probabilidades. Cada valor representa la probabilidad estimada de que el rostro pertenezca a una clase determinada (un alumno). La clase con mayor probabilidad será la elegida como resultado final.

Cálculo de nivel de confianza

Además de la clase predicha, el sistema calcula un indicador de confianza que permite estimar que tan segura es la predicción realizada.

Durante las pruebas realizadas se observó que las predicciones correctas generalmente presentaban niveles de confianza entre el 50% y 70%. Mientras que valores menores correspondían a predicciones poco confiables.

Es por esto que se implementó un porcentaje mínimo de aceptación para determinar si una predicción debía considerarse válida.

Asociación de datos con una identidad

Una vez que el sistema de reconocimiento determina la identidad probable de un alumno, resulta necesario recuperar la información asociada a dicha persona. Para ello se implementó una base de datos local utilizando **SQLite**, permitiendo almacenar y consultar información académica de los estudiantes directamente desde el dispositivo móvil.

El modelo TensorFlow Lite genera una predicción representada mediante una etiqueta y esta funciona como llave de búsqueda dentro de la base de datos.

Resultado final del reconocimiento

Cuando el sistema concluye el proceso, la aplicación muestra en pantalla:

- Cantidad de rostros detectados.
- Identidad reconocida.
- Nivel de confianza.
- Información adicional obtenida de SQLite.

8. Resultados Experimentales y Evaluación del Sistema

Conjunto de prueba

Para garantizar una buena evaluación, el desempeño final modelo fue medido utilizando exclusivamente el subconjunto de prueba. Es decir, aquellas imágenes que no participaron en ninguna etapa del entrenamiento ni la validación.

Dicho conjunto de prueba quedó compuesto por aproximadamente 32 imágenes distribuidas entre los 16 alumnos.

Métricas de evaluación

Para analizar el rendimiento del sistema se utilizaron diversas métricas muy usadas en problemas de clasificación.

- **Accuracy:** porcentaje total de predicciones correctas por el modelo.
- **Precision:** indica que proporción de las predicciones asignadas a una clase fueron correctas.
- **Recall:** identificar correctamente los ejemplos pertenecientes a una determinada clase.
- **F1-Score:** combina Precision y Recall en una única métrica.

Resultados obtenidos

La evaluación final sobre el conjunto de prueba obtuvo los siguientes resultados:

Métrica	Valor
Test Accuracy	93.75 %
Macro F1-Score	0.93

La precisión obtenida indica que el modelo clasificó correctamente aproximadamente 30 de las 32 imágenes de prueba.

Estos resultados evidencian que la combinación de MobileNetV2, transfer learning y data augmentation permitió construir un modelo efectivo a pesar del tamaño relativamente pequeño del conjunto de datos.

Evaluación en dispositivos reales

Una vez integrado el modelo dentro de la aplicación flutter, se realizaron pruebas utilizando dispositivos Android físicos.

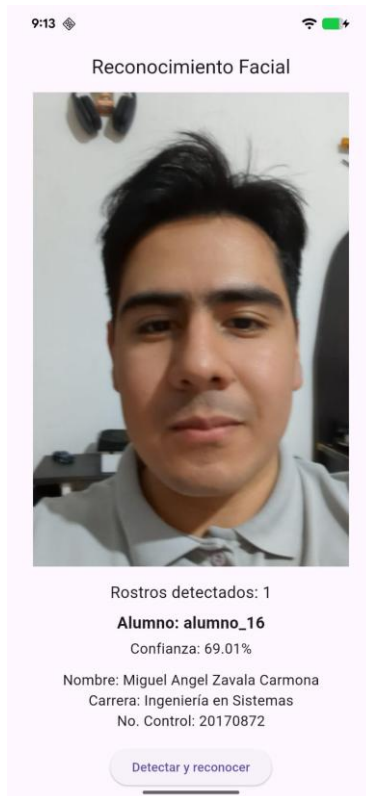
Las pruebas permitieron verificar:

- Captura de imágenes.
- Detección facial con ML Kit.
- Inferencia con TensorFlow Lite.
- Consulta de información en SQLite.
- Presentación de resultados.

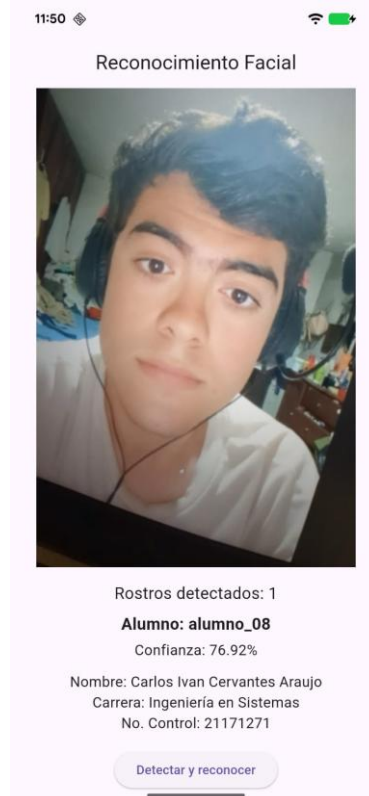
Las siguientes capturas de pantalla muestran al sistema en funcionamiento mostrando la correcta detección de rostros de diferentes alumnos y su información escolar obtenida de la base de datos:



Alumno_07 - Yahir



Alumno_16 - Miguel



Alumno_08 - Carlos

De igual forma, se presentan capturas de pantalla del sistema cuando no detecta correctamente el rostro y no supera el porcentaje de confianza mínimo, o falla en su predicción mostrando información de un alumno que no corresponde:

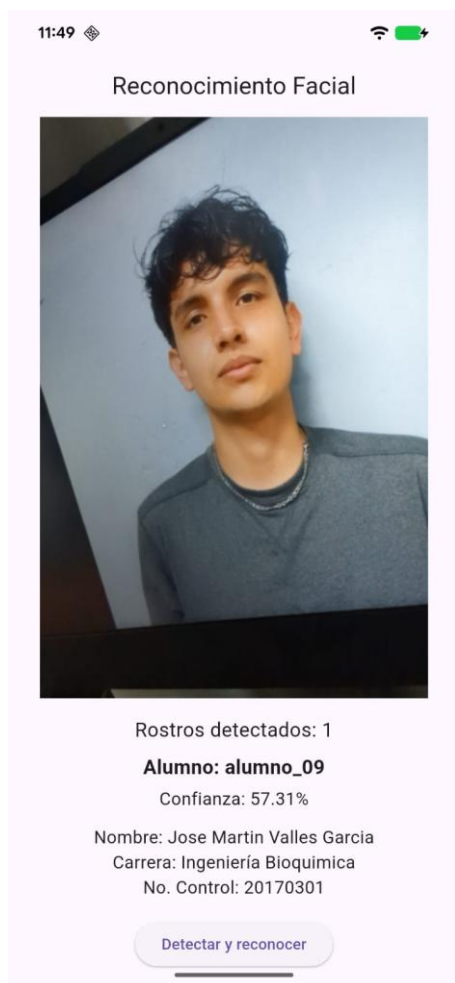


Ilustración 4 Predicción incorrecta



Ilustración 5 Nivel de confianza por debajo del aceptado

Comportamiento observado

Las pruebas mostraron que el sistema detectaba correctamente los rostros frente a la cámara y era capaz de asociarlos con los alumnos registrados en la base de datos.

Al principio se observó que las predicciones presentaban niveles de confianza muy bajos, cercanos al 25%.

Tras analizar el problema, se realizaron ajustes en la interpretación de las probabilidades generadas por TensorFlow Lite y de esta forma mejoraron significativamente los resultados obtenidos.

Las predicciones comenzaron a presentar niveles de confianza aproximadamente entre 50% - 60%.

Estos resultados fueron aceptables tomando en cuenta las diferencias existentes entre las imágenes del Dataset y las condiciones de una captura en tiempo real.

9. Conclusiones

A partir de los resultados obtenidos puede concluirse que los objetivos planteados fueron alcanzados de forma satisfactoria. Se logró construir un proceso completo desde la adquisición y procesamiento de imágenes hasta la identificación de alumnos y recuperación de información asociada en una base de datos.

La utilización de transfer learning mediante MobileNetV2 demostró ser una decisión adecuada para las características del problema planteado. Gracias a esto fue posible desarrollar un clasificador de rostros eficiente sin requerir grandes cantidades de datos ni recursos computacionales excesivos.

El proyecto permitió consolidar conocimientos relacionados con visión artificial, redes neuronales convolucionales, aprendizaje por transferencia y procesamiento digital de imágenes.